TRABAJO PRÁCTICO INTEGRADOR

Sistema de Gestión de Inventario de PCs, Componentes de Hardware y Revisiones Técnicas

Programación II | Licenciatura en Ciencias de la Computación | Año 2026

Integrantes	Entrega y repositorio
Moya Juan Ignacio Candela Julieta Soto	Proyecto_Java_Inventario_PC_POO.zip github.com/Ignaci016/Trabajo-Pr-ctico-Integrador-Programaci-n-2.git

1. Herramientas y tecnologías empleadas

El sistema está desarrollado en Java SE 17 y utiliza SQLite 3 como base de datos local (inventario_tech.db). La conexión se realiza con sqlite-jdbc 3.45.1.0; Maven administra la dependencia y el proyecto fue organizado para trabajar en Apache NetBeans IDE. La aplicación se distribuye en siete subpaquetes: interfaces, exceptions, model, persistence, service, ui y util, además del paquete raíz que contiene Main.

2. Diagrama UML del modelo de dominio

El diagrama refleja las clases, interfaces, atributos principales y relaciones implementadas en el código Java. ProductoInventario es la clase abstracta raíz; EquipoComputo y ComponenteHardware son sus especializaciones abstractas; y cinco clases concretas completan la jerarquía.

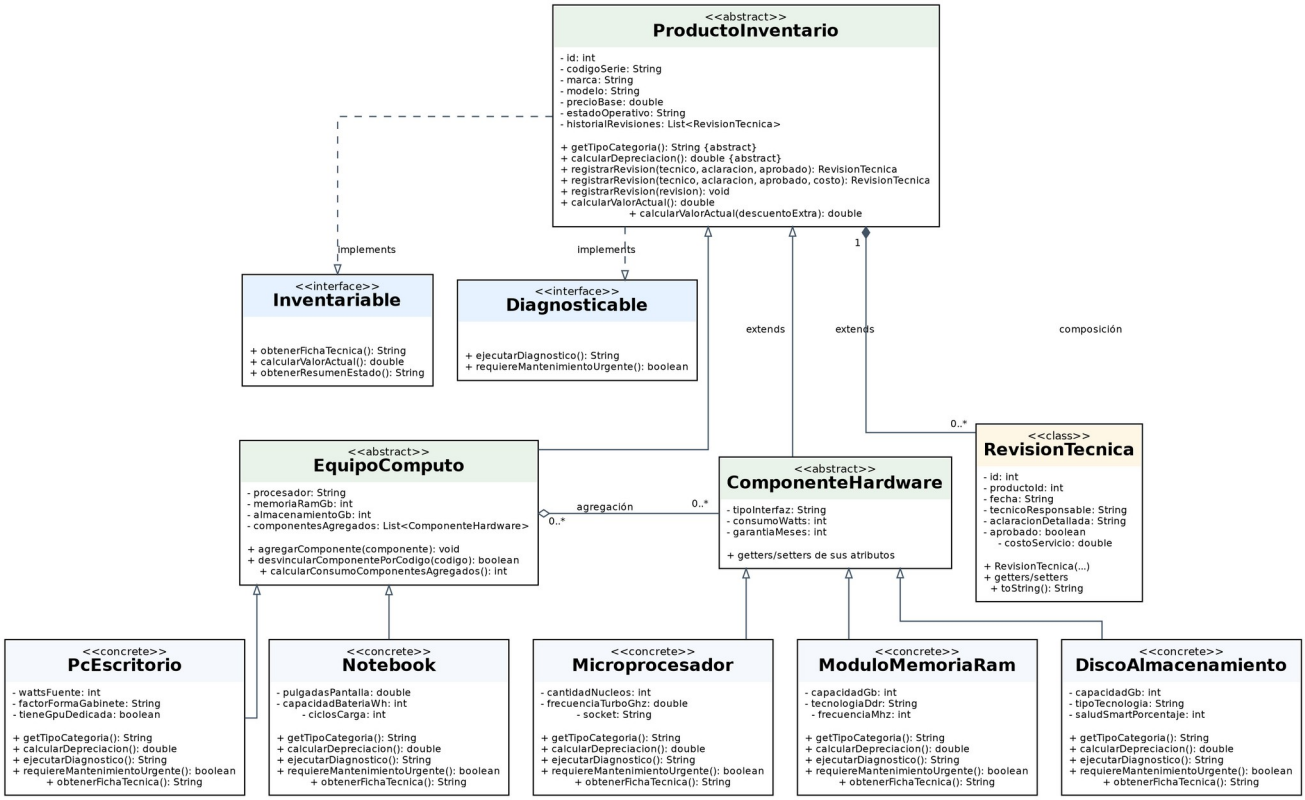


Figura 1. UML del dominio: herencia, interfaces, composición con RevisionTecnica y agregación entre equipos y componentes.

ProductoInventario implementa Inventariable y Diagnosticable. La composición establece que cada RevisionTecnica pertenece a un producto; la agregación se representa mediante una tabla de vínculos que admite múltiples asociaciones en ambos sentidos (0..*).

3. Arquitectura de la aplicación y organización de clases

La arquitectura separa la interacción con el usuario, la lógica de negocio y el acceso a datos. Main inicia la aplicación y construye la interfaz; ConsolaUI solicita operaciones a InventarioService; el servicio coordina las reglas y utiliza ProductoDAO; el DAO ejecuta SQL a través de ConexionSQLite y utiliza ProductoFactory para reconstruir el tipo concreto de cada registro.

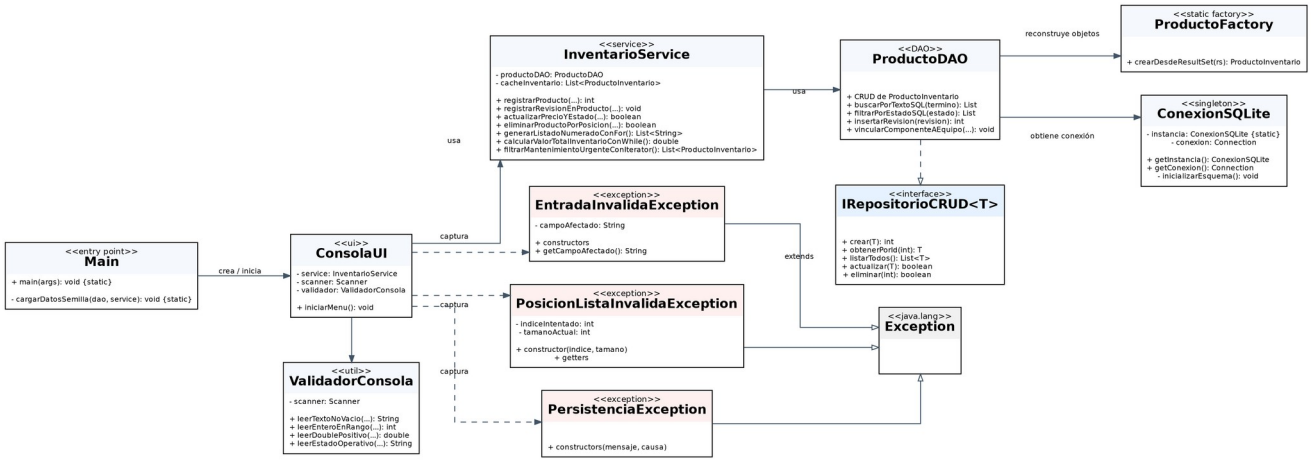


Figura 2. UML de arquitectura: clases de interfaz, servicio, persistencia, factoría, conexión y excepciones.

Paquete	Responsabilidad
model	Entidades del dominio: productos, equipos, componentes y revisiones técnicas.
interfaces	Contratos Inventariable, Diagnosticable e IRepositorioCRUD<T>.
persistence	Conexión SQLite, DAO y factoría de construcción de objetos.
service	Reglas de negocio, coordinación del inventario y recorridos de listas.
ui	Menú de consola y comunicación con el usuario.
util	Lectura y validación centralizada de datos de entrada.
exceptions	Excepciones propias para errores de entrada, posición y persistencia.

4. Decisiones de diseño y relaciones entre clases

Concepto	Aplicación y justificación
Herencia y abstracción	ProductoInventario concentra datos comunes. EquipoComputo y ComponenteHardware agregan atributos de cada familia. PcEscritorio, Notebook, Microprocesador, ModuloMemoriaRam y DiscoAlmacenamiento son clases concretas. Esto evita duplicar atributos y permite especializar comportamientos.
Interfaces y polimorfismo	Inventariable define ficha técnica, valor actual y resumen de estado; Diagnosticable define diagnóstico y urgencia. Las clases concretas implementan su comportamiento. calcularValorActual() utiliza calcularDepreciacion() abstracto y obtiene un resultado según el tipo real del producto.
Sobrecarga	ProductoInventario ofrece tres firmas de registrarRevision(): sin costo explícito, con costo y recibiendo una RevisionTecnica existente. También posee dos versiones de calcularValorActual(): normal y con descuento adicional.
Composición	ProductoInventario mantiene su historial de RevisionTecnica. La revisión tiene productoId obligatorio y la clave foránea usa ON DELETE CASCADE; al eliminar el producto se eliminan sus revisiones.
Agregación	EquipoComputo mantiene una lista de ComponenteHardware. Los componentes existen independientemente en el inventario. equipo_componentes almacena los vínculos; al eliminar un equipo se elimina la asociación, no el componente.

Comportamientos de diagnóstico: PcEscritorio evalúa el margen de potencia de la fuente y el estado operativo; Notebook recomienda reemplazo por encima de 500 ciclos y marca urgencia por encima de 600; RAM aplica depreciación de 5% para DDR5, 10% para DDR4 y 20% para otras tecnologías, y compara umbrales de frecuencia DDR5/DDR4; DiscoAlmacenamiento advierte por salud S.M.A.R.T. inferior al 75%.

5. Persistencia en SQLite y operaciones CRUD

La base de datos contiene tres tablas relacionadas. productos almacena los datos comunes y la categoría de cada producto; revisiones_tecnicas conserva el historial asociado a cada producto; equipo_componentes registra las asociaciones entre equipos y componentes. ProductoDAO implementa crear, obtener por ID, listar, actualizar y eliminar, además de buscar por serie, marca o modelo con WHERE LIKE y filtrar por estado. Las consultas parametrizadas utilizan PreparedStatement.

Elemento	Implementación
ConexionSQLite	Patrón Singleton: constructor privado, instancia única y getInstancia(). Inicializa el esquema y activa PRAGMA foreign_keys = ON.
ProductoDAO	Implementa IRepositorioCRUD<ProductoInventario> y encapsula las sentencias SQL y las operaciones de persistencia.
ProductoFactory	Factoría estática (Simple Factory): analiza tipo_categoria y construye la subclase concreta correspondiente.
Datos iniciales	Main carga los datos de ejemplo únicamente cuando el inventario está vacío; no reinicia la base en cada ejecución.

6. Iteraciones, validación y manejo de excepciones

InventarioService utiliza for para generar el listado numerado, while para sumar el valor total del inventario e Iterator.remove() para filtrar productos que no requieren mantenimiento urgente dentro de una copia de la lista. Ese filtrado no elimina registros de SQLite.

ValidadorConsola centraliza la lectura de texto, enteros en rango, números decimales y estados. EntradaInvalidaException controla datos ingresados incorrectamente; PosicionListaInvalidaException representa índices fuera de rango; y PersistenciaException comunica errores de acceso a la base de datos. ConsolaUI captura los errores y presenta mensajes controlados.

7. Demostración del sistema y pruebas documentadas

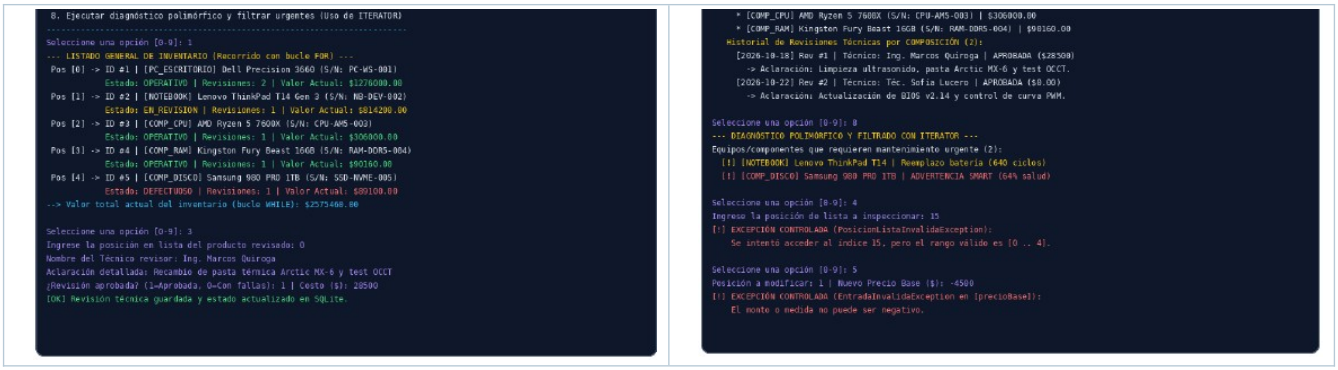


Figura 3. Evidencia de consola: menú y recorridos; registro y consulta de revisiones; ficha técnica, componentes asociados y control de excepciones.

Prueba	Evidencia funcional
CRUD y consultas	El código implementa alta, consulta, modificación, eliminación, búsqueda parametrizada y filtro por estado.
Revisiones	Se registra fecha, técnico, aclaración, aprobación y costo; la revisión se vincula al producto.
Recorridos y errores	Se demuestran for, while, Iterator y excepciones personalizadas para posición inválida e ingreso negativo.